

Guia de Raciocínio — Como Resolver a Pergunta 2 (2a + 2b) Passo a Passo

Este guia ensina-te o **processo mental** para resolver a parte mais difícil do exame de PP: os métodos auxiliares (2a) e o método principal (2b). Não é um template para copiar — é para entenderes **como pensar** quando vês o enunciado pela primeira vez.

Passo 0: Ler o Enunciado com Caneta na Mão

Antes de escreveres uma única linha de código, lê o enunciado da 2a e 2b **completo** e sublinha 3 coisas:

1. **Os inputs** — O que o método recebe? (ex: `Vehicle vehicle`, `AidBox aidbox`)
2. **A regra de negócio** — Qual é a condição? (ex: "tipo igual ao do veículo E medição > 80% da capacidade")
3. **O output** — O que devolve? (ex: `boolean` , `double` , `Route[]` , `String`)

Exemplo Real (Exame 25/26, Pergunta 2a, primeiro método):

"Este método deve devolver `true` caso exista a ocorrência de, pelo menos, um container cujo tipo seja igual ao do veículo e se a sua última medição registada tiver um valor superior a 80% da sua capacidade."

Com a caneta, sublinha:

- **Input:** `Vehicle vehicle`, `AidBox aidbox`
- **Regra:** tipo igual ao do veículo **E** medição > 80% capacidade
- **Output:** `boolean` (true se encontrar **pelo menos um**)

Passo 1: Identificar "Onde Estão os Dados?"

Olha para os **excertos de código fornecidos** no final do enunciado e responde a esta pergunta:

"Como é que eu, a partir dos inputs que recebi, chego aos dados que preciso para avaliar a condição?"

Cadeia de Navegação (Exemplo Real):

```
Vehicle → .getSupplyType()      → dá-me o ItemType do veículo
AidBox  → .getContainers()      → dá-me o Container[]
```

```
Container → .getType()           → dá-me o ItemType do contentor
Container → .getCapacity()        → dá-me a capacidade máxima
Container → .getLastMeasurement() → dá-me a Measurement (pode ser NULL!)
Measurement → .getValue()        → dá-me o valor medido
```

Agora já sabes o **caminho** que tens de percorrer no código. É como um mapa:

```
Vehicle → supplyType
        |
        | comparar com ▼
        |
AidBox → containers[] → cada container → tipo (comparar)
                                         → capacidade
                                         → lastMeasurement → valor
```

Passo 2: Construir o Método Auxiliar (2a) — Do Exterior para o Interior

Agora que sabes o caminho, constrói o código **camada por camada**, de fora para dentro.

Camada 1: A Assinatura e a Proteção

Começa sempre pelo esqueleto mais básico — a assinatura do método e as validações dos parâmetros:

```
public boolean hasCollectableContainer(Vehicle vehicle, AidBox aidbox) {
    // Pergunta: "E se me passarem null?" → Proteger logo aqui
    if (vehicle == null || aidbox == null) return false;

    // ... o resto vem a seguir
}
```

Raciocínio: Se o veículo ou a caixa forem nulos, não há nada para analisar. Devolver `false` imediatamente.

Camada 2: Chegar ao Array

O próximo passo é **extrair o array** que precisas de percorrer. Neste caso, os contentores da AidBox:

```
Container[] containers = aidbox.getContainers();
// Pergunta: "E se a AidBox não tiver contentores?" → Proteger
if (containers == null) return false;
```

Raciocínio: Pediste os contentores. E se o array vier `null` ? Proteges antes de iterar.

Camada 3: O Ciclo (Percorrer o Array)

Agora iteras sobre cada posição do array:

```

for (int i = 0; i < containers.length; i++) {
    // Pergunta: "E se esta posição do array for null?" → Saltar
    if (containers[i] == null) continue;

    // ... verificações vêm aqui dentro
}

// Se chegou aqui, nenhum contentor cumpriu os critérios
return false;

```

Raciocínio: Arrays podem ter posições nulas (especialmente quando geridos manualmente). O `continue` salta para o próximo sem crashar.

Camada 4: Aplicar a Regra de Negócio

Agora, dentro do ciclo, aplicas as condições que sublinhaste no enunciado. **Uma condição de cada vez:**

```

// CONDIÇÃO 1: "tipo igual ao do veículo"
if (containers[i].getType() == vehicle.getSupplyType()) {

    // Agora preciso da medição. MAS PODE SER NULL!
    Measurement last = containers[i].getLastMeasurement();

    if (last != null) {
        // CONDIÇÃO 2: "medição > 80% da capacidade"
        if (last.getValue() > containers[i].getCapacity() * 0.8) {
            return true; // "pelo menos um" → basta encontrar 1
        }
    }
}
}

```

Raciocínio passo a passo:

1. Primeiro verifico o tipo — é a condição mais barata (comparação de enum com `==`)
2. Só se o tipo bater é que vou buscar a medição — porquê desperdiçar tempo se o tipo nem é o certo?
3. Antes de aceder ao `.getValue()`, **verifico se a medição não é null** — este é o ponto onde 80% dos alunos rebentam com `NullPointerException`
4. Só depois faço o cálculo matemático
5. O enunciado diz "pelo menos um" → assim que encontro um que cumpre, faço `return true` imediatamente

Resultado Final Completo:

```

public boolean hasCollectableContainer(Vehicle vehicle, AidBox aidbox) {
    if (vehicle == null || aidbox == null) return false;

    Container[] containers = aidbox.getContainers();
    if (containers == null) return false;

    for (int i = 0; i < containers.length; i++) {
        if (containers[i] == null) continue;

```

```

        if (containers[i].getType() == vehicle.getSupplyType()) {
            Measurement last = containers[i].getLastMeasurement();
            if (last != null) {
                if (last.getValue() > containers[i].getCapacity() * 0.8) {
                    return true;
                }
            }
        }
    }
    return false;
}

```



Passo 3: O Segundo Método da 2a (Ação com Exceção)

O segundo método da 2a é quase sempre um método que **tenta fazer uma ação** e pode falhar.

Ler o Enunciado:

"Para adicionar a AidBox à rota, deve previamente validar com `validator.validate(...)` . Se validado, adicionar com `route.addAidBox(...)` . Caso origine uma `RouteException` , retornar `false` ."

Raciocínio em 3 Perguntas:

1. "Tenho de validar antes?" → Sim, com `validator.validate(route, aidbox)`
2. "O que faço se validar?" → Chamar `route.addAidBox(aidbox)`
3. "O que faço se der erro?" → O `addAidBox` lança `RouteException` → `try-catch` → `return false`

Construção:

```

public boolean addAidBoxToRoute(Route route, AidBox aidbox, RouteValidator validator) {
    // Proteção
    if (route == null || aidbox == null || validator == null) return false;

    // Pergunta 1: Validar primeiro
    if (validator.validate(route, aidbox)) {
        // Pergunta 2: Tentar a ação
        try {
            route.addAidBox(aidbox);
            return true; // Correu bem
        } catch (RouteException e) {
            // Pergunta 3: Se falhar, retornar false
            return false;
        }
    }
}

```

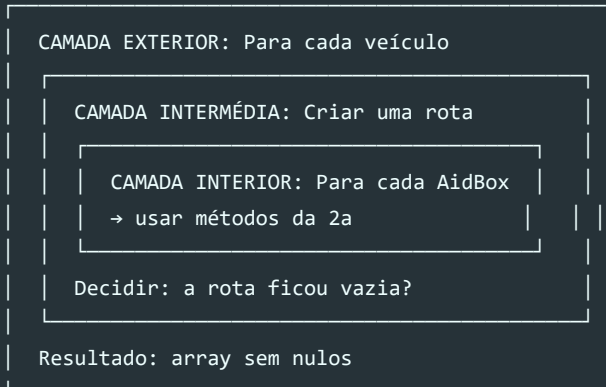
```
return false; // Validação falhou
}
```

Nota importante: O `try-catch` é **obrigatório** aqui porque `addAidBox` declara `throws RouteException` (é uma *checked exception*). Se não tiveres o `try-catch`, o código **não compila**.



Passo 4: O Método Principal da 2b — Pensar em "Camadas de Iteração"

A 2b é o método grande. O truque é pensar nele como **camadas concêntricas**:



Raciocínio Passo a Passo:

1. **"De onde vêm os dados?"** → De `inst.getVehicles()` e `inst.getAidBoxes()`
2. **"Qual é a estrutura lógica?"** → Para **cada veículo**, crio uma rota e tento preencher com `AidBoxes`
3. **"Quantas rotas vou ter?"** → No máximo, tantas quantos os veículos. Mas pode ser menos (se alguma rota ficar vazia).
4. **"Como guardo as rotas sem saber quantas serão?"** → Array temporário com o tamanho máximo + contador:

```
Route[] temp = new Route[vehicles.length]; // Espaço máximo
int count = 0;                             // Quantas tenho realmente
```

5. **"Como sei se uma rota ficou vazia?"** → Uso uma flag `boolean temConteudo = false;` que mudo para `true` quando adiciono algo.

6. **"Como devolvo o array sem nulos?"** → No fim, crio um novo array com o tamanho exato do `count` e copio:

```
Route[] result = new Route[count];
for (int i = 0; i < count; i++) {
    result[i] = temp[i];
}
return result;
```

Construção Completa com Comentários de Raciocínio:

```
@Override
public Route[] generate(IIInstitution inst, RouteValidator validator) {
    // "E se me passarem null?" → array vazio
    if (inst == null || validator == null) return new Route[0];

    // "De onde vêm os dados?"
    Vehicle[] vehicles = inst.getVehicles();
    AidBox[] aidBoxes = inst.getAidBoxes();
    if (vehicles == null || aidBoxes == null) return new Route[0];

    // "Quantas rotas vou ter no máximo?" → tantas quantos veículos
    Route[] temp = new Route[vehicles.length];
    int count = 0;

    // CAMADA EXTERIOR: para cada veículo
    for (int i = 0; i < vehicles.length; i++) {
        if (vehicles[i] == null) continue;

        // CAMADA INTERMÉDIA: criar a rota deste veículo
        Route rota = new RouteImpl(vehicles[i]);
        boolean temConteúdo = false;

        // CAMADA INTERIOR: tentar cada AidBox
        for (int j = 0; j < aidBoxes.length; j++) {
            if (aidBoxes[j] == null) continue;

            // "Esta caixa tem contentores para recolher por este veículo?"
            if (hasCollectableContainer(vehicles[i], aidBoxes[j])) {
                // "Consigo adicionar à rota?"
                if (addAidBoxToRoute(rota, aidBoxes[j], validator)) {
                    temConteúdo = true;
                }
            }
        }

        // "A rota ficou vazia?" → só guardar se tiver conteúdo
        if (temConteúdo) {
            temp[count] = rota;
            count++;
        }
    }

    // "Como devolvo sem nulos?" → array com tamanho exato
    Route[] result = new Route[count];
    for (int i = 0; i < count; i++) {
        result[i] = temp[i];
    }
    return result;
}
```

Passo 5: E se o Método Devolver `String` em vez de `Route[]` ?

O raciocínio é o **mesmo**, mas em vez de criar rotas, constróis uma String:

```
@Override
public String generate(IInstitution inst) {
    // Proteção
    if (inst == null) return "";
    AidBox[] boxes = inst.getAidBoxes();
    if (boxes == null) return "";

    // Em vez de Route[] temp, uso String
    String report = "";

    // CAMADA: para cada AidBox
    for (int i = 0; i < boxes.length; i++) {
        if (boxes[i] == null) continue;

        // "Cumpre a condição?" → usar método da 2a
        double avg = getAverageOccupancy(boxes[i]);
        if (avg > 50.0) {
            // Em vez de adicionar à rota, concateno à String
            report += "Code: " + boxes[i].getCode() + "\n";
            report += "Perishable: " + countContainersByType(boxes[i], ItemType.PERISHABLE_FOOD)
        }
    }

    return report;
}
```

Passo 6: E se o Método Devolver `double` ?

Ainda mais simples — em vez de criar rotas ou Strings, **acumulas um valor**:

```
@Override
public double getTotalCollectedByType(IInstitution inst, ItemType type) {
    if (inst == null || type == null) return 0.0;
    AidBox[] boxes = inst.getAidBoxes();
    if (boxes == null) return 0.0;

    double total = 0.0; // Acumulador

    for (int i = 0; i < boxes.length; i++) {
        if (boxes[i] == null) continue;
        Container[] containers = boxes[i].getContainers();
        if (containers == null) continue;

        for (int j = 0; j < containers.length; j++) {
            if (containers[j] == null) continue;
        }
    }
}
```

```

        // "Cumpra as condições?" → usar métodos da 2a
        if (containers[j].getType() == type && isContainerFull(containers[j], 0.75)) {
            total += getContainerLoad(containers[j]);
        }
    }
}
return total;
}

```

Resumo do Raciocínio em 6 Perguntas

Quando vires a Pergunta 2 no exame, faz estas 6 perguntas mentalmente:

#	Pergunta	Resultado
1	"O que recebo como input?"	Os parâmetros do método
2	"Onde estão os dados que preciso?"	Navegar pelas interfaces fornecidas
3	"Qual é a condição/filtro?"	A regra de negócio do enunciado
4	"O que pode ser NULL no caminho?"	Tudo o que vem de <code>.getAlguma coisa()</code>
5	"Que tipo de resultado devolvo?"	<code>boolean</code> / <code>Route[]</code> / <code>String</code> / <code>double</code>
6	"Preciso de try-catch?"	Sim, se o método chamado declara <code>throws</code>